MIND MATRIX

NAME: MOHAMMED SAHIL R

EMAIL ID: mohammedsahilr39@gmail.com

SESSION: 12

REPORT ON: TASK REPORT

TASK NO: 01-03

TASK 1 – ADVANCED THEMING STRATEGY

(MATERIAL 3 + BRAND IDENTITY)

Introduction

This task focuses on designing a complete theming system for a Compose application using Material Design 3 principles. The goal is to create consistent UI styling by defining color schemes, typography, and shapes that reflect the brand identity of the application.

Theming Concepts

ColorScheme

Defines the main color palette of an application including primary, secondary, background, surface, and error colors.

Typography

Defines font styles used in the application such as headings, subtitles, and body text.

Shapes

Defines the corner radius and shapes used for UI components such as buttons and cards.

Dynamic Color

Dynamic color automatically adapts the app theme based on the user's wallpaper (Android 12+).

Light/Dark Theme

Allows the application UI to adjust colors depending on system theme settings.

Example Color Scheme

```
val LightColors = lightColorScheme(
    primary = Color(0xFF0066CC),
    secondary = Color(0xFFFF9800),
    background = Color(0xFFFFFFFF),
    surface = Color(0xFFF5F5F5)
)
```

Brand Theme Design

App Name: **InspireApp**

Brand Colors

Token	Light Mode	Dark Mode
Primary	#0066CC	#3399FF
Secondary	#FF9800	#FFB74D
Background	#FFFFFF	#121212
Surface	#F5F5F5	#1E1E1E
Error	#B00020	#CF6679

Typography Tokens

Style	Font Size
Heading	24sp
Subtitle	18sp
Body	16sp
Caption	12sp

Shape Tokens

Shape	Size
Small	4dp
Medium	12dp
Large	24dp

Theme Architecture

Files used

- **Color.kt** – color palette
- **Typography.kt** – text styles
- **Shapes.kt** – shape definitions
- **Theme.kt** – MaterialTheme configuration

Example usage

```
MaterialTheme(
    colorScheme = LightColors,
    typography = AppTypography,
    shapes = AppShapes
) {
    AppContent()
}
```

Component Theme Mapping

Component	Theme Usage
AppBar	colorScheme.primary
Card	shapes.medium
Button	colorScheme.secondary

Reflection

A structured theming system ensures UI consistency across the entire application. Without a centralized theme, developers may use random colors and styles, leading to inconsistent design. Supporting light and dark modes improves accessibility and user comfort.

TASK 2 – ANIMATION DESIGN & PLANNING

Introduction

This task focuses on planning animations that enhance user interaction and visual feedback in Compose UI.

Animation APIs

animateFloatAsState
Animates a float value when its state changes.

animateColorAsState
Animates color transitions smoothly.

AnimatedVisibility
Shows or hides UI components with animation.

animateContentSize
Automatically animates size changes in a layout.

Transition
Manages multiple animations between states.

Animation Example

```
val scale by animateFloatAsState(
    targetValue = if (expanded) 1.2f else 1f
)
```

Feature Selected

Feature: **Expandable Quote Card**

Animation helps improve UX by smoothly expanding a card when a user taps it to view more information.

Animation Plan

Scenario	Trigger	Animation	UI Element
Card Expand	Tap card	animateContentSize	Card
Favourite Toggle	Tap icon	animateColorAsState	Icon
Image Fade	Card open	AnimatedVisibility	Image

Performance Considerations

- Avoid heavy animations in large lists
- Keep animation duration short

Accessibility Considerations

- Provide option to reduce motion
- Ensure important information is always visible

Reflection

Animations make UI interactions more intuitive and engaging. However, excessive animations may reduce performance and distract users. Proper planning ensures animations improve usability without affecting performance.

TASK 3 – THEME + ANIMATION INTEGRATION MODULE

Module Description

Module: **Expandable Quote Card Module**

This module displays a list of quotes where each card can expand to show more information and toggle favourite state with animated transitions.

Data Model

```
data class Quote(
    val title: String,
    val author: String,
    val description: String,
    var isFavourite: Boolean
)
```

State Variables

isExpanded
isFavourite
currentThemeMode
animationProgress

UI Composition Plan

MaterialTheme
|
LazyColumn
|
Card
|
Image
Text
Favourite Icon

Animation integration

- expand card → animateContentSize
- favourite icon → animateColorAsState

Implementation Timeline

Step 1 – Apply theme system
Step 2 – Create static UI layout
Step 3 – Add animations
Step 4 – Test responsiveness and performance

Quality Checklist

- Theme tokens used correctly
- Animations smooth
- Images optimized
- UI responsive across devices

Edge Case Tests

Case	Expected Result
Theme change during animation	UI updates smoothly
Rapid favourite toggling	State remains consistent

Reflection

Combining theming and animation creates visually appealing and consistent interfaces. Developers must balance aesthetics with performance to maintain smooth UI experiences.

TASK 4 – CAPSTONE FEATURE IMPLEMENTATION

Implementation Overview

Feature: **Animated Quote List**

The feature includes themed UI components with animated card expansion and favourite toggling.

Implementation Notes

The application uses MaterialTheme for styling and Compose animation APIs for smooth UI transitions.

Example animation snippet

```
AnimatedVisibility(visible = expanded) {
   Text(description)
}
```

Screenshots / Demonstration

- Light theme screenshot
- Dark theme screenshot
- Animation preview

Performance Evaluation

Animations were smooth with minimal UI lag. Future optimization may include reducing animation duration for slower devices.

Presentation Slides

Slide 1 – Feature Overview
Slide 2 – Theme Design & Screenshots
Slide 3 – Animation Flow

Reflection

Designing themed animations required careful planning of UI states and transitions. The most challenging part was maintaining performance while adding visual effects. In future projects, additional features such as search, filters, and dynamic color adaptation can be added.